

Implementation Writeup: Triangular Leading Edges for OpenRocket Fins

M. NICHITIU — MIT ROCKET TEAM SIMULATIONS — 23 JUNE 2026

Contents

1	Abstract	1
2	Feature Summary	1
	2.1 User-Facing Behavior	
	2.2 Core Modeling Decision	
	2.3 Parameterization	
3	Performance Metric	3
4	Implementation Architecture	3
	4.1 Subsystems Modified	
	4.2 Data Flow	
5	File-by-File Modification Rationale	4
6	Core Model: FinSet.java	5
	6.1 New Constants and Enum Value	
	6.2 Stored Property	
	6.3 Angle and Distance Methods	
	6.4 Mass and Volume	
7	Aerodynamic Implementation: FinSetCalc.java	8
	7.1 Additional Geometry State	
	7.2 Roll Forcing	
	7.3 Roll-Projected Chord Integration	
	7.4 Roll Damping	
	7.5 Pressure Drag	
8	Swing UI Implementation	13
	8.1 Shared Controls in FinSetConfig.java	
	8.2 Preview Drawing	
	8.3 Hooking the Shared Controls into Each Dialog	
9	File Format and External Compatibility	16
	9.1 Native .ork Save Path	
	9.2 Native .ork Load Path	
	9.3 File Specification	
	9.4 RockSim Export	
	9.5 RASAero Export	
10	Localization	17
11	Tests	17
	11.1 Aerodynamic Tests	
	11.2 Persistence Test	
12	Validation Commands	19
13	Behavioral Boundaries and Known Tradeoffs	20
	13.1 Not a Full Airfoil	
	13.2 Mass Approximation	
	13.3 Supersonic Attached-Shock Limit	
	13.4 Transonic Blend	
	13.5 External Format Degradation	
14	Conclusion	21

1 Abstract

*This document describes the implementation of triangular leading-edge fin cross sections in OpenRocket. The implemented feature adds a fourth fin cross-section type, **TRIANGULAR**, defined as the existing square-plate section with a symmetric triangular leading-edge wedge. The feature touches the core fin model, Barrowman fin aerodynamics, OpenRocket file persistence, external-format degradation behavior, Swing configuration dialogs, developer file-format documentation, localization strings, and unit tests.*

The feature is parameterized by the full included angle between the top and bottom leading-edge bevel surfaces. The Swing UI also exposes the geometrically equivalent distance from the sharp leading-edge vertex back to the original square-section leading face. The triangular leading-edge projected area is included in fin roll forcing and roll damping calculations, while normal-force planform behavior remains based on the existing fin planform.

The implementation summarized here corresponds to commit [0fadcd](#) of MDNICH/openrocket.

2 Feature Summary

2.1 User-Facing Behavior

The new behavior is visible in the fin configuration dialogs for trapezoidal, elliptical, and freeform fin sets:

- The fin cross-section selector now includes **Triangular leading edge**.
- When the triangular cross section is selected, an additional leading-edge editor appears.
- The editor allows one parameter to be edited at a time: either the included angle or the vertex distance.
- A schematic preview is repainted dynamically as the selected value changes.
- The preview is an isosceles triangular nose attached to the rectangular fin section; the vertical base of the triangle is the fin thickness and the nose vertex is upstream of the original square leading face.

2.2 Core Modeling Decision

The triangular leading-edge section is intentionally not modeled as a complete airfoil. It is modeled as a square plate whose leading edge has been beveled into a symmetric triangular wedge. This matters because:

- The trailing edge remains blunt, so base drag follows the existing square-section behavior.
- The fin planform used for normal-force, CP, and CNa calculations remains unchanged.
- The leading-edge wedge changes pressure drag, volume/mass, and roll-projected area.
- External formats that only understand square, rounded, or airfoil sections can degrade the new option to square rather than pretending it is a rounded or airfoil section.

2.3 Parameterization

Let t be fin thickness, ϕ be the full included leading-edge angle, and d be the distance from the sharp leading-edge vertex to the original square-section leading face. The cross section is an isosceles triangle at the leading edge, so the half-angle relation is

$$\tan\left(\frac{\phi}{2}\right) = \frac{t/2}{d}. \quad (2.3.1)$$

The implementation therefore uses

$$d = \frac{t/2}{\tan(\phi/2)}, \quad (2.3.2)$$

$$\phi = 2 \tan^{-1}\left(\frac{t/2}{d}\right). \quad (2.3.3)$$

The stored model property is ϕ , in radians. The `.ork` field stores the same angle in degrees, matching the surrounding file-format convention for fin angles.

3 Performance Metric

The following comparison was provided for the Zephyrus Test Launch configuration using a less powerful motor. It is included here as an implementation-level sanity metric, not as a formal validation suite.

Tool	Motor	Apogee	Time to Apogee
RAS Aero	N8406-P Motor (MITRT)	11620 ft	24.24 s
OpenRocket	N8406-P Motor (MITRT)	11758 ft	25.70 s
Difference, OpenRocket minus RAS Aero	N8406-P Motor (MITRT)	+138 ft	+1.46 s
Relative difference	N8406-P Motor (MITRT)	+1.19%	+6.02%

The apogee comparison is close in absolute terms for a rocket-flight simulation: OpenRocket predicts an apogee 138 ft higher than RAS Aero on an approximately 11600 ft flight. The time-to-apogee difference is larger in relative terms and should be treated as a reminder that detailed trajectory agreement depends on the motor model, drag buildup, atmospheric assumptions, launch setup, and time integration details, not only on fin leading-edge pressure drag.

4 Implementation Architecture

4.1 Subsystems Modified

The implementation is split across the same boundaries OpenRocket already uses:

- **Core rocket component model:** stores the new cross-section type and leading-edge angle, computes angle/distance conversions, and adjusts volume for the removed leading-edge wedge.
- **Core aerodynamics:** applies the triangular leading-edge shape to pressure drag and roll moments.
- **File I/O:** persists the new cross-section enum value and optional leading-edge angle in OpenRocket `.ork` files.
- **External export:** maps unsupported triangular sections to the closest available external representation.
- **Swing UI:** exposes the new cross section and leading-edge editor in existing fin configuration dialogs.
- **Documentation and tests:** documents the new `.ork` field and verifies the main behavior.

4.2 Data Flow

The feature data flow is:

1. A user selects `TRIANGULAR` from the existing fin cross-section selector.
2. The Swing triangular leading-edge panel becomes visible.
3. The user edits either included angle or vertex distance.
4. Both UI modes call into `FinSet`; the stored value is always `leadingEdgeAngle`.
5. `FinSetCalc` snapshots the cross-section type, thickness, and leading-edge angle when calculating fin aerodynamics.
6. `FinSetSaver` writes the triangular section and non-automatic angle to `.ork`.
7. `DocumentConfig` reads the angle back from `.ork` in degrees and converts it to radians.

5 File-by-File Modification Rationale

File	What changed	Why it changed
core/src/main/java/info/openrocket/core/rocketcomponent/FinSet.java	Added <code>CrossSection.TRIANGULAR</code> ; added <code>leadingEdgeAngle</code> ; added angle/distance conversion helpers; copied the angle during component copy; adjusted fin volume with <code>getFinVolumeFactor()</code> .	This is the authoritative fin data model. It must own the persistent property and provide geometry helpers so the UI, aero code, tests, and file I/O do not duplicate trigonometry.
core/src/main/java/info/openrocket/core/aerodynamics/barrowman/FinSetCalc.java	Added roll-projected area/chord fields; included triangular projected area in roll forcing and damping; added triangular leading-edge pressure-drag calculation; treated triangular base drag as square.	This is where fin forces and moments are computed. The triangular leading edge affects leading-edge pressure drag and roll-projected area, but not the base drag of the blunt trailing edge.
core/src/main/java/info/openrocket/core/file/openrocket/savers/FinSetSaver.java	Writes triangular cross-section XML through the enum name and writes an optional leading-edge angle in degrees.	The OpenRocket native file format must preserve the user's selected triangular section and explicit angle.
core/src/main/java/info/openrocket/core/file/openrocket/importt/DocumentConfig.java	Registers a setter for <code>FinSet.leadingedgeangle</code> , converting degrees to radians.	Loading must be symmetric with saving and consistent with existing angle-field conventions.
core/src/main/java/info/openrocket/core/file/TipShapeCode.java	Maps triangular cross sections to RockSim tip shape code 0.	RockSim has no triangular-leading-edge-only tip shape. Square is the closest compatible fallback because the trailing edge remains blunt.
core/src/main/java/info/openrocket/core/file/rasaero/RASAeroCommonConstants.java	Maps triangular cross sections to RASAero square with a warning.	RASAero export does not support this cross-section type. Warning avoids silent semantic loss.
core/src/main/resources/l10n/messages.properties	Adds English display text, labels, radio-button text, and tooltips for the triangular leading-edge UI.	Swing UI labels are localized through the existing translation infrastructure.
swing/src/main/java/info/openrocket/swing/gui/configdialog/FinSetConfig.java	Adds shared triangular leading-edge controls and preview.	The UI should be implemented once and reused by all fin config dialogs that share the same cross-section model.
swing/src/main/java/info/openrocket/swing/gui/configdialog/TrapezoidFinSetConfig.java	Calls the shared triangular leading-edge control hook.	Trapezoidal fins use the same <code>FinSet</code> cross-section model and need the new controls.
swing/src/main/java/info/openrocket/swing/gui/configdialog/EllipticalFinSetConfig.java	Calls the shared triangular leading-edge control hook.	Elliptical fins use the same <code>FinSet</code> cross-section model and need the new controls.
swing/src/main/java/info/openrocket/swing/gui/configdialog/FreeformFinSetConfig.java	Calls the shared triangular leading-edge control hook.	Freeform fins use the same <code>FinSet</code> cross-section model and need the new controls.
docs/source/dev_guide/file_specification.rst	Documents <code>triangular</code> as a valid <code><crosssection></code> value and documents <code><leadingedgeangle></code> .	The native file format changed, so the developer file specification must describe the new value and units.
core/src/test/java/info/openrocket/core/aerodynamics/FinSetCalcTest.java	Adds tests for pressure drag, base drag, angle/distance conversion, and roll damping area.	The highest-risk behavior is aerodynamic. Tests guard both numerical sanity and intended square-trailing-edge behavior.
core/src/test/java/info/openrocket/core/file/openrocket/OpenRocketSaverTest.java	Adds a save/load round-trip test for triangular cross section and leading-edge angle.	The model must survive <code>.ork</code> persistence without losing the explicit user setting.

6 Core Model: FinSet.java

6.1 New Constants and Enum Value

FinSet is the correct owner for the leading-edge property because every standard fin type derives from it and because cross-section selection already lives there. The new constants define the automatic angle and the hard upper bound:

- DEFAULT_LEADING_EDGE_ANGLE = 20 degrees.
- MAX_LEADING_EDGE_ANGLE = 90 degrees.

The enum value TRIANGULAR is given a relative volume of 1.00 in the enum itself because the true volume adjustment depends on chord, thickness, and angle. That angle-aware correction is applied later by `getFinVolumeFactor()`.

```

40 // Triangular leading edges are parameterized by the full included angle
41 // between the two beveled surfaces. A stored angle of zero uses this default.
42 public static final double DEFAULT_LEADING_EDGE_ANGLE = Math.toRadians(20);
43 public static final double MAX_LEADING_EDGE_ANGLE = Math.PI / 2;

45 /**
46  * Maximum number of root points in the root geometry.
47  */
48 private static final int MAX_ROOT_DIVISIONS = 100;
49 private static final int MAX_ROOT_DIVISIONS_LOW_RES = MAX_ROOT_DIVISIONS / 5;

51 public void setOverrideMass() {
52 }

54 public enum CrossSection {
55     /// Square
56     SQUARE(trans.get("FinSet.CrossSection.SQUARE"), 1.00),
57     /// Rounded
58     ROUNDED(trans.get("FinSet.CrossSection.ROUNDED"), 0.99),
59     /// Airfoil
60     AIRFOIL(trans.get("FinSet.CrossSection.AIRFOIL"), 0.85),
61     /// Triangular leading edge
62     // Starts from the square-plate section; only the leading edge is beveled.
63     TRIANGULAR(trans.get("FinSet.CrossSection.TRIANGULAR"), 1.00);

```

6.2 Stored Property

The stored property is `leadingEdgeAngle`. It is deliberately stored as an angle rather than a distance:

- A leading-edge included angle is scale-independent and meaningful across fins of different thicknesses.
- The same angle can be applied to different local chord lengths and thicknesses.
- A distance is still exposed in the UI as a derived/editable representation for users who prefer that construction dimension.

```

122 /**
123  * Included angle between the top and bottom surfaces of a triangular leading edge.
124  * Zero means automatic.
125  */
126 private double leadingEdgeAngle = 0;

```

6.3 Angle and Distance Methods

The setter propagates to configuration listeners, matches the pattern used by other `FinSet` fields, and keeps zero as a sentinel for automatic/default behavior. This avoids writing a redundant field for older designs or non-triangular cross sections.

The effective-angle helper also enforces a local-chord constraint. If a requested long wedge would not fit into a short local station near a fin tip, the method increases the angle enough that the triangular nose fits within the available local chord.

```

308 public double getLeadingEdgeAngle() {
309     return leadingEdgeAngle;
310 }

312 public void setLeadingEdgeAngle(double angle) {
313     for (RocketComponent listener : configListeners) {
314         if (listener instanceof FinSet) {
315             ((FinSet) listener).setLeadingEdgeAngle(angle);
316         }
317     }

319     // Keep zero as "automatic" so older designs and non-triangular fins do not
320     // gain a redundant leading-edge parameter.
321     double clamped;
322     if (angle <= 0 && Double.isNaN(angle)) {
323         clamped = 0;

```

```

324     } else {
325         clamped = MathUtil.clamp(angle, MathUtil.EPSILON, MAX_LEADING_EDGE_ANGLE);
326     }
327     if (MathUtil.equals(leadingEdgeAngle, clamped))
328         return;
329     leadingEdgeAngle = clamped;
330     fireComponentChangeEvent(ComponentChangeEvent.BOTH_CHANGE);
331 }

333 public double getEffectiveLeadingEdgeAngle() {
334     return getEffectiveLeadingEdgeAngle(Double.POSITIVE_INFINITY);
335 }

337 public double getEffectiveLeadingEdgeAngle(double localChord) {
338     return getEffectiveLeadingEdgeAngle(leadingEdgeAngle, thickness, localChord);
339 }

341 public double getLeadingEdgeDistance() {
342     return leadingEdgeDistanceFromAngle(thickness, getEffectiveLeadingEdgeAngle());
343 }

345 public void setLeadingEdgeDistance(double distance) {
346     if (distance <= 0 Double.isNaN(distance)) {
347         setLeadingEdgeAngle(0);
348     } else {
349         setLeadingEdgeAngle(leadingEdgeAngleFromDistance(thickness, distance));
350     }
351 }

353 public static double getEffectiveLeadingEdgeAngle(double angle, double thickness, double localChord) {
354     double effective = angle > 0 ? angle : DEFAULT_LEADING_EDGE_ANGLE;
355     effective = MathUtil.clamp(effective, MathUtil.EPSILON, MAX_LEADING_EDGE_ANGLE);
356     if (thickness <= MathUtil.EPSILON localChord <= MathUtil.EPSILON Double.isInfinite(localChord)) {
357         return effective;
358     }
359     // If the local chord is shorter than the requested bevel distance, increase
360     // the angle so the triangular nose still fits within the fin planform station.
361     double minimum = 2 * Math.atan((thickness / 2) / (0.5 * localChord));
362     return MathUtil.clamp(Math.max(effective, minimum), MathUtil.EPSILON, MAX_LEADING_EDGE_ANGLE);
363 }

365 public static double leadingEdgeDistanceFromAngle(double thickness, double angle) {
366     if (thickness <= MathUtil.EPSILON) {
367         return 0;
368     }
369     // For an isosceles triangular nose, tan(phi/2) = (thickness/2) / distance.
370     double effective = MathUtil.clamp(angle > 0 ? angle : DEFAULT_LEADING_EDGE_ANGLE,
371         MathUtil.EPSILON, MAX_LEADING_EDGE_ANGLE);
372     return (thickness / 2) / Math.tan(effective / 2);
373 }

375 public static double leadingEdgeAngleFromDistance(double thickness, double distance) {
376     if (thickness <= MathUtil.EPSILON distance <= MathUtil.EPSILON) {
377         return 0;
378     }
379     return MathUtil.clamp(2 * Math.atan((thickness / 2) / distance),
380         MathUtil.EPSILON, MAX_LEADING_EDGE_ANGLE);
381 }

```

6.4 Mass and Volume

The mass model previously used `crossSection.getRelativeVolume()` directly. For triangular leading edges the removed material is a function of the bevel distance and the mean chord, so a new helper computes the correction. The approximation treats the triangular section as a square plate with the leading-edge wedge removed. For a mean chord c and wedge distance d , the removed fraction is approximated as

$$f_{\text{removed}} = \frac{1}{2} \frac{d}{c}, \quad (6.4.1)$$

clamped so the factor remains bounded.

```

1755 private void calculateCM(){
1756     final CoordinateIF wettedCentroid = calculateSinglePlanformCentroid();
1757     this.singlePlanformArea = wettedCentroid.getWeight();
1758     final double wettedVolume = wettedCentroid.getWeight() * thickness * getFinVolumeFactor();
1759     final double finBulkMass = wettedVolume * material.getDensity();
1760     final CoordinateIF wettedCM = wettedCentroid.setWeight(finBulkMass);

1762     final CoordinateIF tabCentroid = calculateTabCentroid();
1763     final double tabVolume = tabCentroid.getWeight() * thickness;
1764     final double tabMass = tabVolume * material.getDensity();
1765     final CoordinateIF tabCM = tabCentroid.setWeight(tabMass);

```

```

1767     CoordinateIF filletCentroid = calculateFilletVolumeCentroid();
1768     double filletVolume = filletCentroid.getWeight();
1769     double filletMass = filletVolume * filletMaterial.getDensity();
1770     final CoordinateIF filletCM = filletCentroid.setWeight(filletMass);

1772     this.totalVolume = (wettedVolume + tabVolume + filletVolume) * finCount;

1774     final double eachFinMass = finBulkMass + tabMass + filletMass;
1775     final CoordinateIF eachFinCenterOfMass = wettedCM.average(tabCM).average(filletCM).setWeight(eachFinMass)
;

1777     // ^^ per fin
1778     // vv per component

1780     // set y coordinate: rotate around parent, if single fin; otherwise multiple fins will average out to
zero
1781     if (finCount == 1) {
1782         this.centerOfMass = baseRotation.transform( eachFinCenterOfMass );
1783     } else {
1784         this.centerOfMass = eachFinCenterOfMass.setY(0.0).setWeight( eachFinMass * this.finCount);
1785     }
1786 }

1788 private double getFinVolumeFactor() {
1789     if (crossSection != CrossSection.TRIANGULAR) {
1790         return crossSection.getRelativeVolume();
1791     }
1792     if (singlePlanformArea <= MathUtil.EPSILON thickness <= MathUtil.EPSILON) {
1793         return 1.0;
1794     }
1795     // Triangular sections are modeled as square fins with the leading-edge wedge
1796     // removed. The factor is an area-weighted approximation based on mean chord.
1797     double meanChord = getLength();
1798     if (meanChord <= MathUtil.EPSILON) {
1799         meanChord = singlePlanformArea / Math.max(getSpan(), MathUtil.EPSILON);
1800     }
1801     double angle = getEffectiveLeadingEdgeAngle(meanChord);
1802     double bevelLength = leadingEdgeDistanceFromAngle(thickness, angle);
1803     double removedFraction = MathUtil.clamp(bevelLength / meanChord, 0, 1) / 2;
1804     return 1 - removedFraction;
1805 }

```

7 Aerodynamic Implementation: FinSetCalc.java

7.1 Additional Geometry State

FinSetCalc previously used one local chord array for all fin integrations. The triangular leading edge required separating planform chord from roll-projected chord:

- `chordLength`: existing planform chord, retained for normal-force and pitch/yaw behavior.
- `rollChordLength`: chord used in roll forcing and damping, including leading-edge wedge projection.
- `rollProjectedArea`: integrated roll-projected area.
- `rollMacSpan`: spanwise centroid of roll-projected area.

```

39     protected double rollProjectedArea = Double.NaN; // Area seen by roll flow, including triangular leading-edge
        projection.
40     protected double rollMacSpan = Double.NaN; // Spanwise center of the roll-projected area.

42     protected int interferenceFinCount = -1; // No. of fins in interference

44     protected double[] chordLead = new double[DIVISIONS];
45     protected double[] chordTrail = new double[DIVISIONS];
46     protected double[] chordLength = new double[DIVISIONS];
47     protected double[] rollChordLength = new double[DIVISIONS]; // Local chord used in roll damping/forcing
        integrals.

49     protected final WarningSet geometryWarnings = new WarningSet();

51     private final double[] poly = new double[6];

53     private final double thickness;
54     private final double bodyRadius;
55     private final int finCount;
56     private final double cantAngle;
57     private final FinSet.CrossSection crossSection;
58     private final double leadingEdgeAngle;

```

7.2 Roll Forcing

Roll forcing uses the roll-projected area and its centroid. This is the implementation point for the requirement that the fin leading-edge area be included in total projected area for aerodynamic roll moments. The normal-force planform stays unchanged; only the roll moment arm and area scaling are adjusted.

```

172         // Calculate roll forces, reduce forcing above stall angle

174         // Without body-fin interference effect:
175         //     forces.CrollForce = fins * (macSpan+r) * cnal * component.getCantAngle() /
176         //     conditions.getRefLength();
177         // With body-fin interference effect:
178         // Triangular leading edges add projected area ahead of the rectangular plate.
179         // Use that area for roll moments without changing the normal-force planform.
180         double rollAreaRatio = finArea > MathUtil.EPSILON ? rollProjectedArea / finArea : 1;
181         forces.setCrollForce((rollMacSpan + r) * cnal * rollAreaRatio * (1 + tau) * cantAngle /
182             conditions.getRefLength());

```

7.3 Roll-Projected Chord Integration

The roll chord at each span station is

$$c_{\text{roll}}(y) = c_{\text{planform}}(y) + \min(d(y), c_{\text{planform}}(y)). \quad (7.3.1)$$

The cap avoids unrealistic extra projected length at very short local chord stations near tips. The roll-projected area and centroid are integrated across the existing DIVISIONS spanwise strips.

```

345     rollSum = 0;
346     double area = 0;
347     rollProjectedArea = 0;
348     double radius = component.getFinFront().getY();

350     final double dy = span / (DIVISIONS - 1);
351     for (int i = 0; i < DIVISIONS; i++) {
352         double length = chordTrail[i] - chordLead[i];
353         // Roll uses the physical projection of the leading-edge wedge; pitch/yaw
354         // terms keep using the original planform chord.
355         rollChordLength[i] = calculateRollProjectedChordLength(chordLength[i]);
356         double y = i * dy;

358         macLength += length * length;
359         macSpan += y * length;

```

```

360     rollMacSpan += y * rollChordLength[i];
361     macLead += chordLead[i] * length;
362     area += length;
363     rollProjectedArea += rollChordLength[i];
364     rollSum += rollChordLength[i] * pow2(radius + y);

366     if (i > 0) {
367         double dx = (chordTrail[i] + chordLead[i]) / 2 - (chordTrail[i - 1] + chordLead[i - 1]) / 2;
368         double hypot = MathUtil.hypot(dx, dy);
369         if (hypot != 0) {
370             cosGamma += dy / hypot;
371         }

373         dx = chordLead[i] - chordLead[i - 1];
374         hypot = MathUtil.hypot(dx, dy);
375         if (hypot != 0) {
376             cosGammaLead += dy / hypot;
377         }
378     }
379 }

381 macLength *= dy;
382 //logger.debug("macLength = {}", macLength);
383 macSpan *= dy;
384 rollMacSpan *= dy;
385 macLead *= dy;
386 area *= dy;
387 rollProjectedArea *= dy;
388 rollSum *= dy;
389 if (area > MathUtil.EPSILON) {
390     macLength /= area;
391     macSpan /= area;
392     macLead /= area;
393 } else {
394     macLength = 0;
395     macSpan = 0;
396     macLead = 0;
397 }
398 if (rollProjectedArea > MathUtil.EPSILON) {
399     rollMacSpan /= rollProjectedArea;
400 } else {
401     rollProjectedArea = area;
402     rollMacSpan = macSpan;
403 }
404 cosGamma /= (DIVISIONS - 1);
405 cosGammaLead /= (DIVISIONS - 1);
406 }

408 private double calculateRollProjectedChordLength(double planformChord) {
409     if (crossSection != FinSet.CrossSection.TRIANGULAR
410         planformChord <= MathUtil.EPSILON thickness <= MathUtil.EPSILON) {
411         return planformChord;
412     }
413     // The extra projected area is the distance from the square section's leading
414     // face to the triangular nose vertex, capped by the local chord near tips.
415     double angle = FinSet.getEffectiveLeadingEdgeAngle(leadingEdgeAngle, thickness, planformChord);
416     double bevellength = FinSet.leadingEdgeDistanceFromAngle(thickness, angle);
417     bevellength = MathUtil.clamp(bevellength, 0, planformChord);
418     return planformChord + bevellength;
419 }

```

7.4 Roll Damping

The same projected chord is used in both low-speed/high-rate and supersonic roll damping branches. This keeps the roll damping implementation consistent with the roll forcing implementation.

```

506     if (absRate * (bodyRadius + span) / conditions.getVelocity() > 15 * Math.PI / 180) {
507         double sum = 0;
508         for (int i = 0; i < DIVISIONS; i++) {
509             double dist = bodyRadius + span * i / DIVISIONS;
510             double aoa = Math.min(absRate * dist / conditions.getVelocity(), 15 * Math.PI / 180);
511             sum += rollChordLength[i] * dist * aoa;
512         }
513         sum = sum * (span / DIVISIONS) * 2 * Math.PI / conditions.getBeta() /
514             (conditions.getRefArea() * conditions.getRefLength());

516         return MathUtil.sign(rollRate) * sum;
517     }

519     if (mach <= CNA_SUBSONIC) {
520         return 2 * Math.PI * rollRate * rollSum /
521             (conditions.getRefArea() * conditions.getRefLength() *
522              conditions.getVelocity() * conditions.getBeta());
523     }

```

```

524     if (mach >= CNA_SUPERSONIC) {
525         double vel = conditions.getVelocity();
526         double k1 = K1.getValue(mach);
527         double k2 = K2.getValue(mach);
528         double k3 = K3.getValue(mach);

530         double sum = 0;

532         for (int i = 0; i < DIVISIONS; i++) {
533             double y = i * span / (DIVISIONS - 1);
534             double angle = rollRate * (bodyRadius + y) / vel;

536             sum += (k1 * angle + k2 * angle * angle + k3 * angle * angle * angle)
537                   * rollChordLength[i] * (bodyRadius + y);
538         }

```

7.5 Pressure Drag

The pressure-drag implementation has three pieces:

1. The existing rounded-leading-edge correlation was extracted into `calculateRoundedLeadingEdgePressureCoefficient()` so it can be reused.
2. TRIANGULAR dispatches to `calculateTriangularLeadingEdgePressureCoefficient()`.
3. Supersonic wedge pressure is computed with an attached weak-shock solution where possible.

The leading-edge sweep correction is handled by using the leading-edge-normal Mach number

$$M_{\text{edge}} = M \max(0, \cos \gamma_{\text{lead}}). \quad (7.5.1)$$

For a symmetric triangular leading edge, each face is modeled as a wedge with deflection angle

$$\theta = \frac{\phi}{2}. \quad (7.5.2)$$

For $M_{\text{edge}} \leq 0.9$, the implementation reuses the rounded-leading-edge correlation evaluated at M_{edge} . For $M_{\text{edge}} \geq 1.1$, it solves the oblique-shock $\theta - \beta - M$ relation for the weak attached shock:

$$\tan \theta = 2 \cot \beta \frac{M^2 \sin^2 \beta - 1}{M^2 (\gamma + \cos 2\beta) + 2}. \quad (7.5.3)$$

The pressure coefficient is then computed from the normal-shock pressure ratio:

$$\frac{p_2}{p_1} = \frac{2\gamma M_n^2 - (\gamma - 1)}{\gamma + 1}, \quad C_p = \frac{p_2/p_1 - 1}{\frac{1}{2}\gamma M^2}. \quad (7.5.4)$$

If no attached weak-shock root is found, the code falls back to the square-edge stagnation limit. Between Mach 0.9 and 1.1, a smoothstep blend avoids a discontinuity.

```

674         crossSection == FinSet.CrossSection.ROUNDED) {

676             // Round leading edge
677             cd = calculateRoundedLeadingEdgePressureCoefficient(mach);

679         } else if (crossSection == FinSet.CrossSection.SQUARE) {
680             cd = stagnationCD;
681         } else if (crossSection == FinSet.CrossSection.TRIANGULAR) {
682             cd = calculateTriangularLeadingEdgePressureCoefficient(mach, stagnationCD);
683         } else {
684             throw new UnsupportedOperationException("Unsupported fin profile: " + crossSection);
685         }

687         // Slanted leading edge
688         cd *= pow2(cosGammaLead);

690         // Scale to correct reference area
691         cd *= span * thickness / conditions.getRefArea();

693         return cd;
694     }

696     private double calculateRoundedLeadingEdgePressureCoefficient(double mach) {
697         if (mach < 0.9) {
698             return Math.pow(1 - pow2(mach), -0.417) - 1;
699         } else if (mach < 1) {
700             return 1 - 1.785 * (mach - 0.9);
701         } else {

```

```

702         return 1.214 - 0.502 / pow2(mach) + 0.1095 / pow2(pow2(mach));
703     }
704 }

706 private double calculateTriangularLeadingEdgePressureCoefficient(double mach, double stagnationCD) {
707     // Resolve Mach normal to the swept leading edge. A symmetric triangular
708     // leading edge behaves like two equal wedges with deflection phi / 2.
709     double edgeMach = mach * Math.max(0, cosGammaLead);
710     double chord = macLength > MathUtil.EPSILON ? macLength : 0;
711     double phi = FinSet.getEffectiveLeadingEdgeAngle(leadingEdgeAngle, thickness, chord);
712     double theta = phi / 2;

714     if (edgeMach <= 0.9) {
715         // Below transonic speeds, retain the existing rounded-leading-edge
716         // correlation but apply it to the leading-edge-normal Mach number.
717         return calculateRoundedLeadingEdgePressureCoefficient(edgeMach);
718     }
719     if (edgeMach >= 1.1) {
720         // Supersonic wedge drag uses the attached-shock solution when possible;
721         // detached shocks fall back to the square-edge stagnation limit.
722         double cp = calculateAttachedWedgePressureCoefficient(edgeMach, theta);
723         return Double.isNaN(cp) ? stagnationCD : Math.min(cp, stagnationCD);
724     }

726     // Blend through the transonic gap to avoid a discontinuity at Mach 1.
727     double subsonic = calculateRoundedLeadingEdgePressureCoefficient(0.9);
728     double supersonic = calculateAttachedWedgePressureCoefficient(1.1, theta);
729     if (Double.isNaN(supersonic)) {
730         supersonic = stagnationCD;
731     } else {
732         supersonic = Math.min(supersonic, stagnationCD);
733     }
734     double fraction = MathUtil.clamp((edgeMach - 0.9) / 0.2, 0, 1);
735     double smooth = fraction * fraction * (3 - 2 * fraction);
736     return subsonic * (1 - smooth) + supersonic * smooth;
737 }

739 private double calculateAttachedWedgePressureCoefficient(double mach, double theta) {
740     if (mach <= 1 || theta <= 0) {
741         return Double.NaN;
742     }

744     // Solve the theta-beta-M relation for the weak attached shock. No root means
745     // the requested wedge angle exceeds the attached-shock limit for this Mach.
746     double betaMin = Math.asin(1 / mach) + 1.0e-8;
747     double betaMax = Math.PI / 2 - 1.0e-8;
748     double previousBeta = betaMin;
749     double previousValue = thetaFromShockAngle(mach, previousBeta) - theta;

751     for (int i = 1; i <= 200; i++) {
752         double beta = betaMin + (betaMax - betaMin) * i / 200.0;
753         double value = thetaFromShockAngle(mach, beta) - theta;
754         if (previousValue <= 0 && value >= 0) {
755             double root = bisectShockAngle(mach, theta, previousBeta, beta);
756             double normalMachSquared = pow2(mach * Math.sin(root));
757             double pressureRatio = (2 * GAMMA * normalMachSquared - (GAMMA - 1)) / (GAMMA + 1);
758             return (pressureRatio - 1) / (0.5 * GAMMA * pow2(mach));
759         }
760         previousBeta = beta;
761         previousValue = value;
762     }

764     return Double.NaN;
765 }

767 private double bisectShockAngle(double mach, double theta, double low, double high) {
768     for (int i = 0; i < 80; i++) {
769         double mid = (low + high) / 2;
770         double value = thetaFromShockAngle(mach, mid) - theta;
771         if (value >= 0) {
772             high = mid;
773         } else {
774             low = mid;
775         }
776     }
777     return (low + high) / 2;
778 }

780 private double thetaFromShockAngle(double mach, double beta) {
781     double sinBeta = Math.sin(beta);
782     double numerator = 2 / Math.tan(beta) * (pow2(mach) * pow2(sinBeta) - 1);
783     double denominator = pow2(mach) * (GAMMA + Math.cos(2 * beta)) + 2;
784     return Math.atan(numerator / denominator);
785 }

```

```
787 @Override
788 public double calculateComponentBaseCD(FlightConditions conditions,
789                                       double baseCD, WarningSet warnings) {
790     // a fin with 0 area contributes no drag
791     if (finArea < MathUtil.EPSILON) {
792         return 0.0;
793     }
794
795     double cd = 0;
796
797     // Trailing edge drag. The triangular option only changes the leading edge,
798     // so its blunt trailing edge matches the square-section base drag.
799     if (crossSection == FinSet.CrossSection.SQUARE || crossSection == FinSet.CrossSection.TRIANGULAR) {
800         cd = baseCD;
801     } else if (crossSection == FinSet.CrossSection.ROUNDED) {
802         cd = baseCD / 2;
803     }
804     // Airfoil assumed to have zero base drag
```

8 Swing UI Implementation

8.1 Shared Controls in FinSetConfig.java

The UI implementation lives in `FinSetConfig` rather than in each fin dialog because trapezoid, elliptical, and freeform fins all share the same `FinSet` cross-section model. The method creates:

- a mode selector with `Angle` and `Distance`;
- an angle spinner, unit selector, and slider;
- a distance spinner and unit selector;
- a preview panel;
- a visibility listener that shows the editor only for `TRIANGULAR`.

```

152 protected void addTriangularLeadingEdgeControls(JPanel panel) {
153     FinSet finSet = (FinSet) component;
154     JPanel editor = new JPanel(new MigLayout("fillx, gap rel unrel, ins 0", "[ ][65lp::][30lp::][100lp::]", ""
155 ));
156     editor.setBorder(BorderFactory.createEmptyBorder(0, 12, 0, 0));
157
158     // Angle and vertex distance are two editable views of the same stored angle.
159     JLabel title = new JLabel(trans.get("FinSetConfig.lbl.LeadngEdge"));
160     editor.add(title, "span, wrap");
161
162     JLabel modeLabel = new JLabel(trans.get("FinSetConfig.lbl.LeadngEdgeMode"));
163     editor.add(modeLabel);
164     JRadioButton angleMode = new JRadioButton(trans.get("FinSetConfig.LeadngEdgeMode.Angle"));
165     JRadioButton distanceMode = new JRadioButton(trans.get("FinSetConfig.LeadngEdgeMode.Distance"));
166     ButtonGroup modeGroup = new ButtonGroup();
167     modeGroup.add(angleMode);
168     modeGroup.add(distanceMode);
169     angleMode.setSelected(true);
170     JPanel modePanel = new JPanel(new MigLayout("ins 0, gap rel 0"));
171     modePanel.add(angleMode);
172     modePanel.add(distanceMode);
173     editor.add(modePanel, "span 3, growx, wrap");
174     order.add(angleMode);
175     order.add(distanceMode);
176
177     JLabel angleLabel = new JLabel(trans.get("FinSetConfig.lbl.LeadngEdgeAngle"));
178     angleLabel.setToolTipText(trans.get("FinSetConfig.ttip.LeadngEdgeAngle"));
179     editor.add(angleLabel);
180     DoubleModel angleModel = new DoubleModel(component, "LeadngEdgeAngle", UnitGroup.UNITS_ANGLE,
181         0, FinSet.MAX_LEADING_EDGE_ANGLE);
182     register(angleModel);
183     JSpinner angleSpinner = new JSpinner(angleModel.getSpinnerModel());
184     angleSpinner.setEditor(new SpinnerEditor(angleSpinner));
185     angleSpinner.setToolTipText(trans.get("FinSetConfig.ttip.LeadngEdgeAngle"));
186     editor.add(angleSpinner, "growx");
187     order.add(((SpinnerEditor) angleSpinner.getEditor()).getTextField());
188     UnitSelector angleUnit = new UnitSelector(angleModel);
189     editor.add(angleUnit, "growx");
190     BasicSlider angleSlider = new BasicSlider(angleModel.getSliderModel(0, FinSet.DEFAULT_LEADING_EDGE_ANGLE,
191         FinSet.MAX_LEADING_EDGE_ANGLE));
192     editor.add(angleSlider, "w 100lp, wrap");
193
194     JLabel distanceLabel = new JLabel(trans.get("FinSetConfig.lbl.LeadngEdgeDistance"));
195     distanceLabel.setToolTipText(trans.get("FinSetConfig.ttip.LeadngEdgeDistance"));
196     editor.add(distanceLabel);
197     DoubleModel distanceModel = new DoubleModel(component, "LeadngEdgeDistance", UnitGroup.UNITS_LENGTH, 0);
198     register(distanceModel);
199     JSpinner distanceSpinner = new JSpinner(distanceModel.getSpinnerModel());
200     distanceSpinner.setEditor(new SpinnerEditor(distanceSpinner));
201     distanceSpinner.setToolTipText(trans.get("FinSetConfig.ttip.LeadngEdgeDistance"));
202     editor.add(distanceSpinner, "growx");
203     order.add(((SpinnerEditor) distanceSpinner.getEditor()).getTextField());
204     UnitSelector distanceUnit = new UnitSelector(distanceModel);
205     editor.add(distanceUnit, "growx, wrap");
206
207     LeadingEdgePreview preview = new LeadingEdgePreview(finSet);
208     editor.add(preview, "skip, span 3, growx, h 90lp!, wrap");
209
210     ActionListener updateEditMode = e -> {
211         boolean angleSelected = angleMode.isSelected();
212         setComponentsEnabled(angleLabel, angleSelected);
213         setComponentsEnabled(angleSpinner, angleSelected);
214         setComponentsEnabled(angleUnit, angleSelected);
215         setComponentsEnabled(angleSlider, angleSelected);
216         setComponentsEnabled(distanceLabel, !angleSelected);
217         setComponentsEnabled(distanceSpinner, !angleSelected);
218         setComponentsEnabled(distanceUnit, !angleSelected);
219         preview.repaint();
220     };

```

```

220 angleMode.addActionListener(updateEditMode);
221 distanceMode.addActionListener(updateEditMode);

223 StateChangeListener visibilityListener = new StateChangeListener() {
224     @Override
225     public void stateChanged(EventObject e) {
226         boolean triangular = finSet.getCrossSection() == FinSet.CrossSection.TRIANGULAR;
227         editor.setVisible(triangular);
228         updateEditMode.actionPerformed(null);
229         preview.repaint();
230         JComponent parent = (JComponent) editor.getParent();
231         if (parent != null) {
232             parent.revalidate();
233             parent.repaint();
234         }
235     }
236 };
237 component.addChangeListener(visibilityListener);
238 visibilityListener.stateChanged(new EventObject(component));

240 panel.add(editor, "span, growx, wrap para");

```

8.2 Preview Drawing

The preview is intentionally schematic but preserves the selected included angle. It uses a unit-thickness triangle to compute the correct leading-edge distance for the current angle, then scales the triangle and rectangular fin body into the preview area.

```

247 private static class LeadingEdgePreview extends JPanel implements StateChangeListener {
248     private final FinSet finSet;

250     private LeadingEdgePreview(FinSet finSet) {
251         this.finSet = finSet;
252         setPreferredSize(new Dimension(260, 90));
253         finSet.addChangeListener(this);
254     }

256     @Override
257     public void stateChanged(EventObject e) {
258         repaint();
259     }

261     @Override
262     protected void paintComponent(Graphics g) {
263         super.paintComponent(g);
264         Graphics2D g2 = (Graphics2D) g.create();
265         try {
266             g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

268             int margin = 10;
269             int h = getHeight() - 2 * margin;
270             int w = getWidth() - 2 * margin;
271             int maxBaseHeight = Math.max(2, Math.min(h - 16, 46));
272             double angle = Math.max(finSet.getEffectiveLeadingEdgeAngle(), Math.toRadians(0.25));
273             double leadingEdgeDistance = FinSet.leadingEdgeDistanceFromAngle(1.0, angle);
274             double maxBevellLength = Math.max(8, w * 0.45);
275             // Fit the whole cross-section to the preview, preserving the selected included angle.
276             double scale = Math.min(maxBevellLength / Math.max(leadingEdgeDistance, 0.001), maxBaseHeight);
277             int baseHeight = (int) MathUtil.clamp(Math.round(scale), 2, maxBaseHeight);
278             int bevellLength = (int) MathUtil.clamp(Math.round(leadingEdgeDistance * scale), 2, maxBevellLength);

279             int baseX = margin + bevellLength;
280             int midY = getHeight() / 2;
281             int topY = midY - baseHeight / 2;
282             int bottomY = midY + baseHeight / 2;
283             int rectRight = getWidth() - margin;

285             Path2D triangle = new Path2D.Double();
286             triangle.moveTo(margin, midY);
287             triangle.lineTo(baseX, topY);
288             triangle.lineTo(baseX, bottomY);
289             triangle.closePath();

291             Color fill = new Color(180, 205, 230);
292             Color rectFill = new Color(220, 225, 230);
293             Color stroke = getForeground();

295             g2.setColor(fill);
296             g2.fill(triangle);
297             g2.setColor(rectFill);
298             g2.fillRect(baseX, topY, rectRight - baseX, baseHeight);
299             g2.setColor(stroke);
300             g2.draw(triangle);
301             g2.drawRect(baseX, topY, rectRight - baseX, baseHeight);

```

```

302         g2.drawLine(baseX, topY, baseX, bottomY);
303     } finally {
304         g2.dispose();
305     }
306 }

```

8.3 Hooking the Shared Controls into Each Dialog

Each standard fin configuration dialog adds the shared control block after thickness and cross-section controls. This keeps the triangular-specific UI close to the fin cross-section UI without duplicating the controls.

```

181         panel.add(new UnitSelector(thicknessModel), "growx");
182         panel.add(new BasicSlider(thicknessModel.getSliderModel(0, 0.01)), "w 100lp, wrap para");
183     }

185     addTriangularLeadingEdgeControls(panel);

125         panel.add(new UnitSelector(m), "growx");
126         panel.add(new BasicSlider(m.getSliderModel(0, 0.01)), "w 100lp, wrap 30lp");
127     }

129     addTriangularLeadingEdgeControls(panel);

188         panel.add(new UnitSelector(m), "growx");
189         panel.add(new BasicSlider(m.getSliderModel(0, 0.01)), "w 100lp, wrap 30lp");
190     }

192     addTriangularLeadingEdgeControls(panel);

```

9 File Format and External Compatibility

9.1 Native .ork Save Path

FinSetSaver already writes `<crosssection>` by using the enum name. Adding the `TRIANGULAR` enum value therefore automatically makes the new cross-section string available as `triangular`. The only new save logic needed is the optional angle tag. The tag is omitted when the stored angle is zero, preserving the automatic/default behavior.

```

21     elements.add("<thickness>" + fins.getThickness() + "</thickness>");
22     elements.add("<crosssection>" + fins.getCrossSection().name().toLowerCase(Locale.ENGLISH)
23         + "</crosssection>");
24     // Only triangular leading edges have an extra shape parameter. The .ork
25     // value is written in degrees to match the surrounding fin angle fields.
26     if (fins.getCrossSection() == info.openrocket.core.rocketcomponent.FinSet.CrossSection.TRIANGULAR &&
27         !MathUtil.equals(fins.getLeadingEdgeAngle(), 0)) {
28         elements.add("<leadingedgeangle>" + Math.toDegrees(fins.getLeadingEdgeAngle()) + "</leadingedgeangle>
29     ");
30 }

```

9.2 Native .ork Load Path

DocumentConfig maps `FinSet:leadingedgeangle` to `setLeadingEdgeAngle(double)` with a degrees-to-radians conversion. This follows the existing convention for `cant` and similar angle fields.

```

279     setters.put("FinSet:crosssection", new EnumSetter<>{
280         Reflection.findMethod(FinSet.class, "setCrossSection", FinSet.CrossSection.class),
281         FinSet.CrossSection.class));
282     // .ork stores the triangular leading-edge included angle in degrees.
283     setters.put("FinSet:leadingedgeangle", new DoubleSetter(
284         Reflection.findMethod(FinSet.class, "setLeadingEdgeAngle", double.class), Math.PI / 180.0));

```

9.3 File Specification

The developer file specification documents the new `crosssection` value and the unit semantics of `leadingedgeangle`.

```

673 The “<crosssection>” value for standard fin sets is one of “square“,
674 “rounded“, “airfoil“, or “triangular“. For “triangular“ cross sections,
675 “<leadingedgeangle>” may be written after “<crosssection>“. The value is the
676 included angle, in degrees, between the top and bottom leading-edge bevel
677 surfaces. If omitted, OpenRocket uses its automatic triangular leading-edge
678 angle.

```

9.4 RockSim Export

RockSim tip shape codes do not represent a triangular-leading-edge-only section. Export therefore maps triangular fins to square. This is a deliberate approximation because the implemented triangular section retains a blunt square-like trailing edge.

```

25     public static int convertTipShapeCode(FinSet.CrossSection cs) {
26         if (FinSet.CrossSection.ROUNDED.equals(cs)) {
27             return 1;
28         }
29         if (FinSet.CrossSection.AIRFOIL.equals(cs)) {
30             return 2;
31         }
32         if (FinSet.CrossSection.TRIANGULAR.equals(cs)) {
33             // RockSim has no triangular leading-edge code; square preserves the
34             // same blunt trailing edge and is the closest export shape.
35             return 0;
36         }
37         return 0;
38     }

```

9.5 RASAero Export

RASAero export also degrades triangular fins to square. Unlike the RockSim helper, the RASAero path has a warning set available, so the user is warned about the loss of triangular leading-edge information.

```

325     public static String OPENROCKET_TO_RASAERO_FIN_CROSSSECTION(FinSet.CrossSection crossSection, WarningSet
326         warnings) {
327         if (FinSet.CrossSection.SQUARE.equals(crossSection) || FinSet.CrossSection.TRIANGULAR.equals(crossSection))
328         {
329             if (FinSet.CrossSection.TRIANGULAR.equals(crossSection)) {
330                 // RASAero has no triangular leading-edge fin section, so warn and
331                 // degrade to the closest supported section instead of silently changing it.
332                 String msg = "RASAero export does not support triangular fin leading edges, exporting as Square."
333             ;
334             warnings.add(msg);
335             log.warn(msg);
336         }
337         return CROSS_SECTION_SQUARE;
338     }

```

10 Localization

The Swing UI does not hard-code visible labels. New translation keys were added for the cross-section display name, editor labels, radio-button labels, and tooltips.

```

2544 FinSet.CrossSection.TRIANGULAR = Triangular leading edge
2545 FinSetConfig.lbl.LeadingleEdge = Leading edge
2546 FinSetConfig.lbl.LeadingleEdgeMode = Edit
2547 FinSetConfig.lbl.LeadingleEdgeAngle = Included angle
2548 FinSetConfig.lbl.LeadingleEdgeDistance = Vertex distance
2549 FinSetConfig.LeadingleEdgeMode.Angle = Angle
2550 FinSetConfig.LeadingleEdgeMode.Distance = Distance
2551 FinSetConfig.ttip.LeadingleEdgeAngle = Angle between the top and bottom surfaces of the leading-edge bevel.
2552 FinSetConfig.ttip.LeadingleEdgeDistance = Distance from the sharp leading-edge vertex to the full-thickness fin
      section.

```

11 Tests

11.1 Aerodynamic Tests

FinSetCalcTest adds three coverage points:

- triangular pressure drag is positive in subsonic and supersonic regimes;
- triangular base drag follows square trailing-edge behavior;
- triangular leading-edge projected area increases roll damping relative to square.

```

283 public void testTriangularLeadingEdgePressureAndBaseDrag() {
284     Rocket rocket = TestRockets.makeEstesAlphaIII();
285     TrapezoidFinSet fins = (TrapezoidFinSet) rocket.getChild(0).getChild(1).getChild(0);
286     fins.setCrossSection(FinSet.CrossSection.TRIANGULAR);
287     fins.setLeadingEdgeAngle(Math.toRadians(30));

289     FlightConfiguration config = rocket.getSelectedConfiguration();
290     FlightConditions conditions = new FlightConditions(config);
291     WarningSet warnings = new WarningSet();
292     FinSetCalc calc = new FinSetCalc(fins);

294     conditions.setMach(0.3);
295     double subsonicPressureCD = calc.calculatePressureCD(conditions, 1.0, 0.5, warnings);
296     assertTrue(subsonicPressureCD > 0, "Triangular fin subsonic pressure CD should be positive");

298     conditions.setMach(2.0);
299     double supersonicPressureCD = calc.calculatePressureCD(conditions, 1.0, 0.5, warnings);
300     assertTrue(supersonicPressureCD > 0, "Triangular fin supersonic pressure CD should be positive");
301     assertTrue(supersonicPressureCD < 1.0, "Triangular fin supersonic pressure CD should stay below square
      stagnation scaling");

303     double baseCD = calc.calculateComponentBaseCD(conditions, 0.5, warnings);
304     double expectedBase = 0.5 * fins.getSpan() * fins.getThickness() / conditions.getRefArea();
305     assertEquals(expectedBase, baseCD, EPSILON, "Triangular fin should keep square trailing-edge base drag");
306 }

308 @Test
309 public void testTriangularLeadingEdgeAngleDistanceConversion() {
310     double thickness = 0.003;
311     double angle = Math.toRadians(20);
312     double distance = FinSet.leadingEdgeDistanceFromAngle(thickness, angle);
313     assertEquals(angle, FinSet.leadingEdgeAngleFromDistance(thickness, distance), EPSILON,
314         "Leading-edge angle and distance should round trip");
315 }

317 @Test
318 public void testTriangularLeadingEdgeIncreasesProjectedRollDampingArea() {
319     Rocket rocket = TestRockets.makeEstesAlphaIII();
320     TrapezoidFinSet fins = (TrapezoidFinSet) rocket.getChild(0).getChild(1).getChild(0);

322     FlightConditions conditions = new FlightConditions(rocket.getSelectedConfiguration());
323     conditions.setMach(0.3);
324     conditions.setRollRate(5.0);
325     WarningSet warnings = new WarningSet();

327     fins.setCrossSection(FinSet.CrossSection.SQUARE);
328     AerodynamicForces squareForces = new AerodynamicForces();
329     new FinSetCalc(fins).calculateNonaxialForces(conditions, Transformation.IDENTITY, squareForces, warnings);
;

331     fins.setCrossSection(FinSet.CrossSection.TRIANGULAR);
332     fins.setLeadingEdgeAngle(Math.toRadians(20));
333     AerodynamicForces triangularForces = new AerodynamicForces();
334     new FinSetCalc(fins).calculateNonaxialForces(conditions, Transformation.IDENTITY, triangularForces,
      warnings);

```

```

336         assertTrue(triangularForces.getCrollDamp() > squareForces.getCrollDamp(),
337                     "Triangular leading-edge projected area should contribute to roll damping");
338     }

```

11.2 Persistence Test

OpenRocketSaverTest verifies that both `<crosssection>triangular</crosssection>` and `<leadingedgeangle>` are written, then reloads the file and checks that both the enum and angle survive the round trip.

```

165     public void testTriangularFinLeadingEdgeAngleRoundTrip() throws IOException {
166         Rocket rocket = TestRockets.makeEstesAlphaIII();
167         TrapezoidFinSet fins = (TrapezoidFinSet) rocket.getChild(0).getChild(1).getChild(0);
168         fins.setCrossSection(FinSet.CrossSection.TRIANGULAR);
169         fins.setLeadingEdgeAngle(Math.toRadians(25.0));

171         OpenRocketDocument rocketDoc = OpenRocketDocumentFactory.createDocumentFromRocket(rocket);
172         File file = saveRocket(rocketDoc, new StorageOptions());
173         String xml = Files.readString(file.toPath(), StandardCharsets.UTF_8);

175         assertTrue(xml.contains("<crosssection>triangular</crosssection>"),
176                     "Triangular fin cross-section should be written to .ork XML");
177         assertTrue(xml.contains("<leadingedgeangle>25.0</leadingedgeangle>"),
178                     "Explicit triangular leading-edge angle should be written in degrees");

180         OpenRocketDocument loaded = loadRocket(file.getPath());
181         TrapezoidFinSet loadedFins = (TrapezoidFinSet) loaded.getRocket().getChild(0).getChild(1).getChild(0);
182         assertEquals(FinSet.CrossSection.TRIANGULAR, loadedFins.getCrossSection());
183         assertEquals(Math.toRadians(25.0), loadedFins.getLeadingEdgeAngle(), 1.0e-10);
184     }

```

12 Validation Commands

The implementation was validated with focused compile and test commands:

```
1 ./gradlew core:compileJava swing:compileJava
2 ./gradlew core:test --tests "info.openrocket.core.aerodynamics.FinSetCalcTest"
3 ./gradlew core:test --tests "info.openrocket.core.file.openrocket.OpenRocketSaverTest.
  testTriangularFinLeadingEdgeAngleRoundTrip" swing:test --tests "*FinSetConfig*"
4 git diff --check
```

13 Behavioral Boundaries and Known Tradeoffs

13.1 Not a Full Airfoil

The new cross section is not a complete airfoil. The implemented model only changes the leading edge. This is why triangular base drag remains square-like and why external export falls back to square rather than airfoil.

13.2 Mass Approximation

The volume correction uses a mean-chord approximation. This captures the dominant effect of removing the triangular leading-edge wedge without introducing a full spanwise volume integration into `FinSet.calculateCM()`. The more exact alternative would integrate local chord and local bevel distance across span, but the current implementation keeps the change localized and compatible with the existing mass model.

13.3 Supersonic Attached-Shock Limit

The wedge solver only returns an attached weak-shock solution. For wedge angles too large for an attached shock at the current Mach number, it returns `NaN`; the caller then falls back to the square-edge stagnation limit. This prevents a failed root solve from producing nonphysical negative or discontinuous pressure drag.

13.4 Transonic Blend

The transonic region is not a first-principles transonic solver. It is a smooth interpolation between the subsonic correlation and the supersonic wedge model. This is consistent with the pragmatic empirical style already present in OpenRocket fin aerodynamics.

13.5 External Format Degradation

OpenRocket `.ork` files preserve the new feature. RockSim and RASAero exports do not, because those formats do not expose an equivalent fin section. The implementation chooses square as the least misleading fallback because the triangular option preserves the blunt trailing edge.

14 Conclusion

The triangular leading-edge feature is implemented as a narrowly scoped extension of the existing square fin cross section. The core model owns the parameterization, the aerodynamics layer accounts for pressure drag and roll-projected area, the Swing layer exposes an angle/distance editor and preview, and the native file format preserves the feature. Unsupported external formats degrade explicitly to square. The tests cover pressure drag, base drag, roll damping, angle/distance conversion, and `.ork` persistence.

AI Usage

We acknowledge use of AI in the development of this document and the codebase it describes. For more information contact nichitiu@mit.edu or mnichiti@cern.ch.